

Kubernetes: Authentication

VISHWANATH M S
VISHWACLOUDLAB.IN

Our Agenda



- 01 RBAC in Kubernetes
- 02 Why RBAC ?
- 03 Benefits of using RBAC
- 04 Implementing RBAC
- 05 Demo

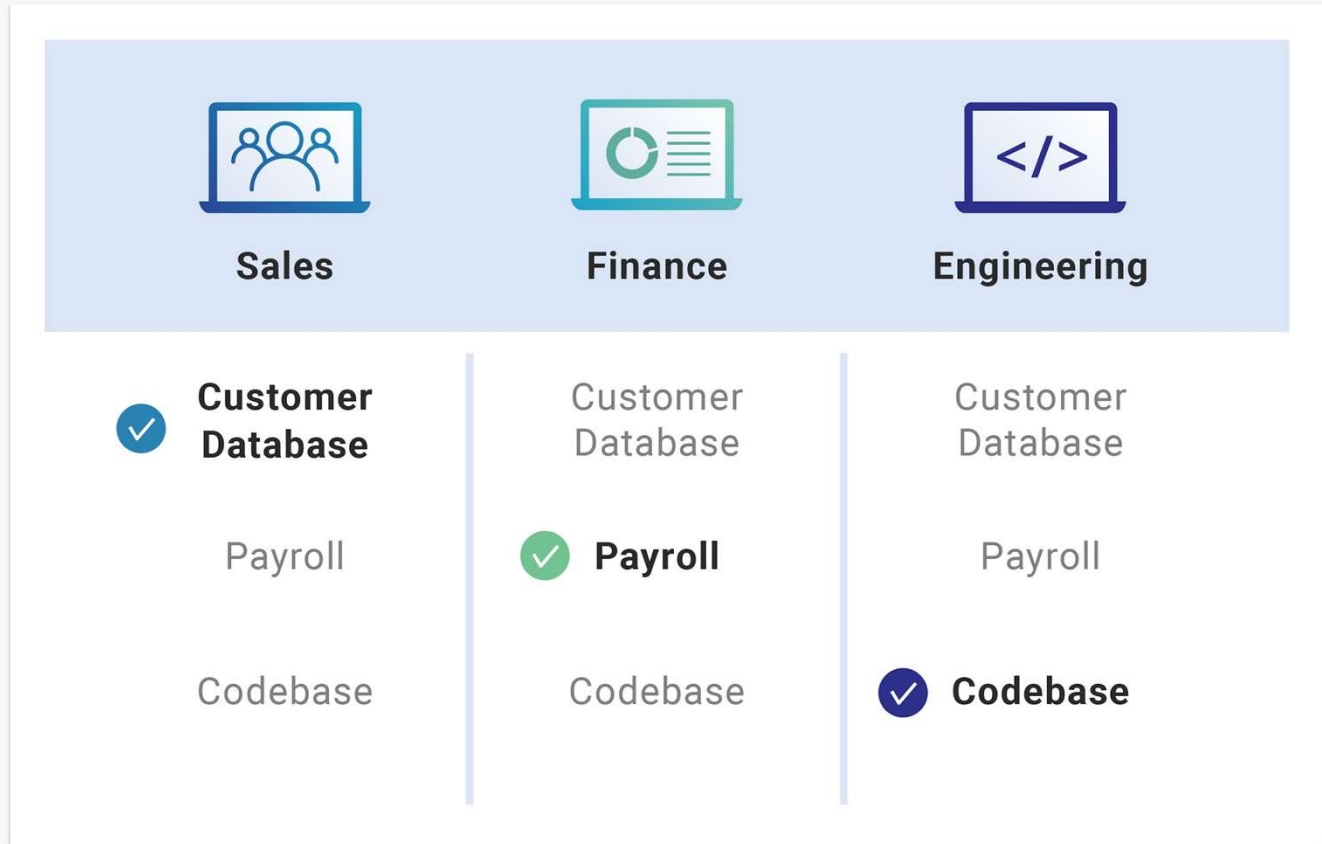
RBAC in our daily life



Roles of different individuals

Each individual in today's world, is assigned a specific role and is expected to work in the allotted field, for example like HR, Finance, Engineering, Sales etc. without hampering the work of any other individual belonging to any other domain.

This brings out the need for some access control policy, in order to restrict the permissions allotted to a user.



RBAC in Kubernetes

Introduction



- Role-based access control (RBAC) is a method of regulating access to computer or network resources based on the roles of individual users within your organization.
- RBAC is used by Kubernetes for authorization, for example giving access to a user or an application/tool, adding/removing permissions and setting up rules, etc.
- So basically, RBAC in K8s, adds security to a Kubernetes cluster.
- RBAC in Kubernetes is the way that you restrict who can access what within the cluster.



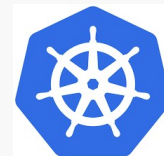
Why RBAC ?

The motivation behind using RBAC

- Let's try to understand this with a use-case scenario,
 - Well obviously, you have multiple people on your teams and they're accessing your Kubernetes cluster.
 - Each of them needs to have some amount of security from one another. There can be cases, where a member in your team is interfering with the other member's work, accidentally, for example, one developer might accidentally delete someone else's work or gain visibility into a secret project that's under development and not open for an audience yet.
 - In such cases, having a Role-Based Access Control is mandatory.

Before vs. After

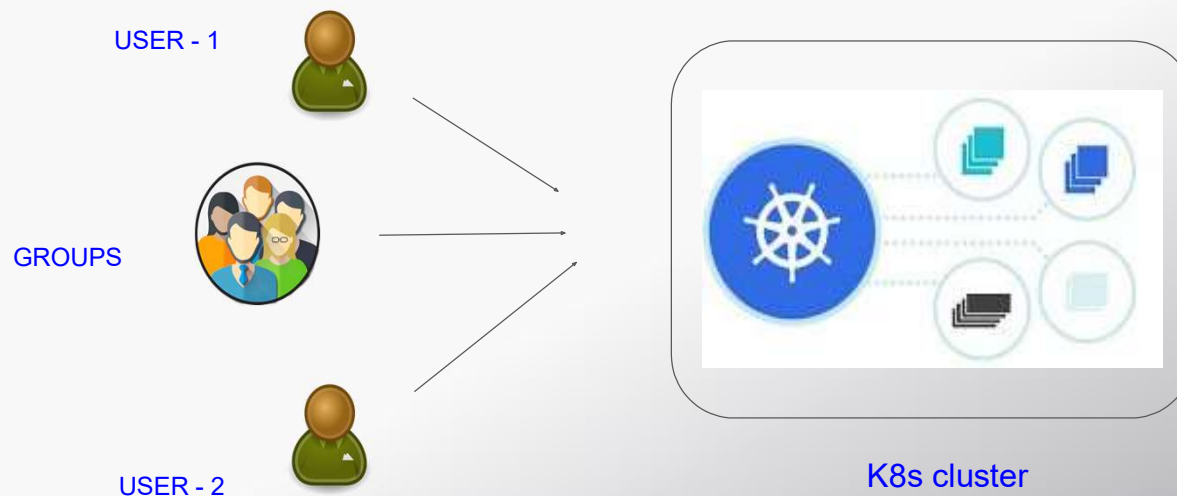
RBAC



BEFORE RBAC :

Here, the User-1 has the complete access to the K8s cluster resources.

This can affect the work of other users or groups of users in the same namespace or maybe in different namespaces.



BEFORE RBAC

BAD
IDEA!!!

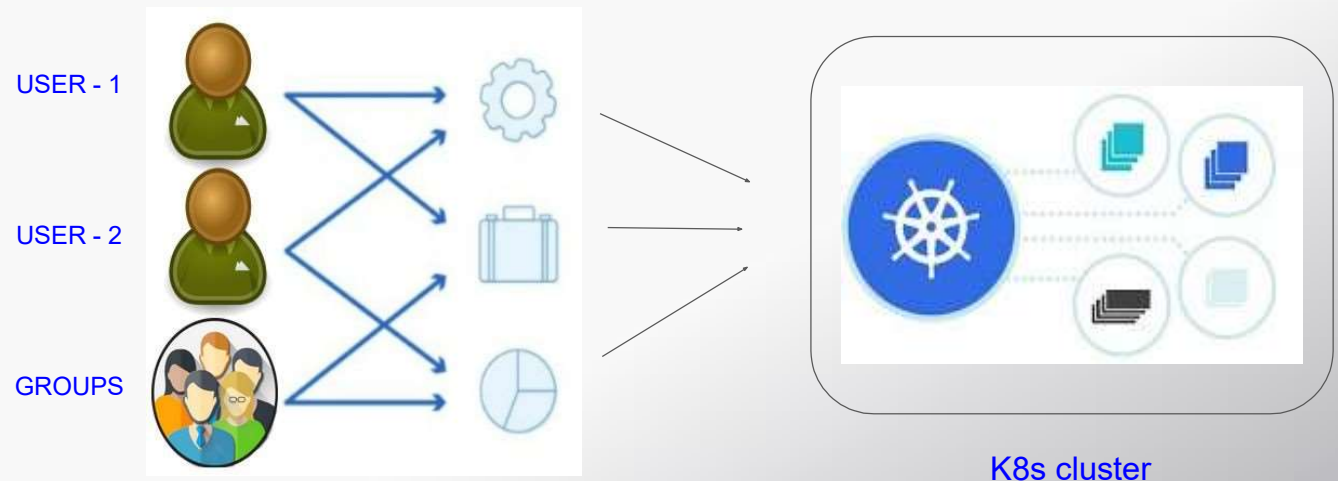


Before vs. After

After RBAC

AFTER RBAC :

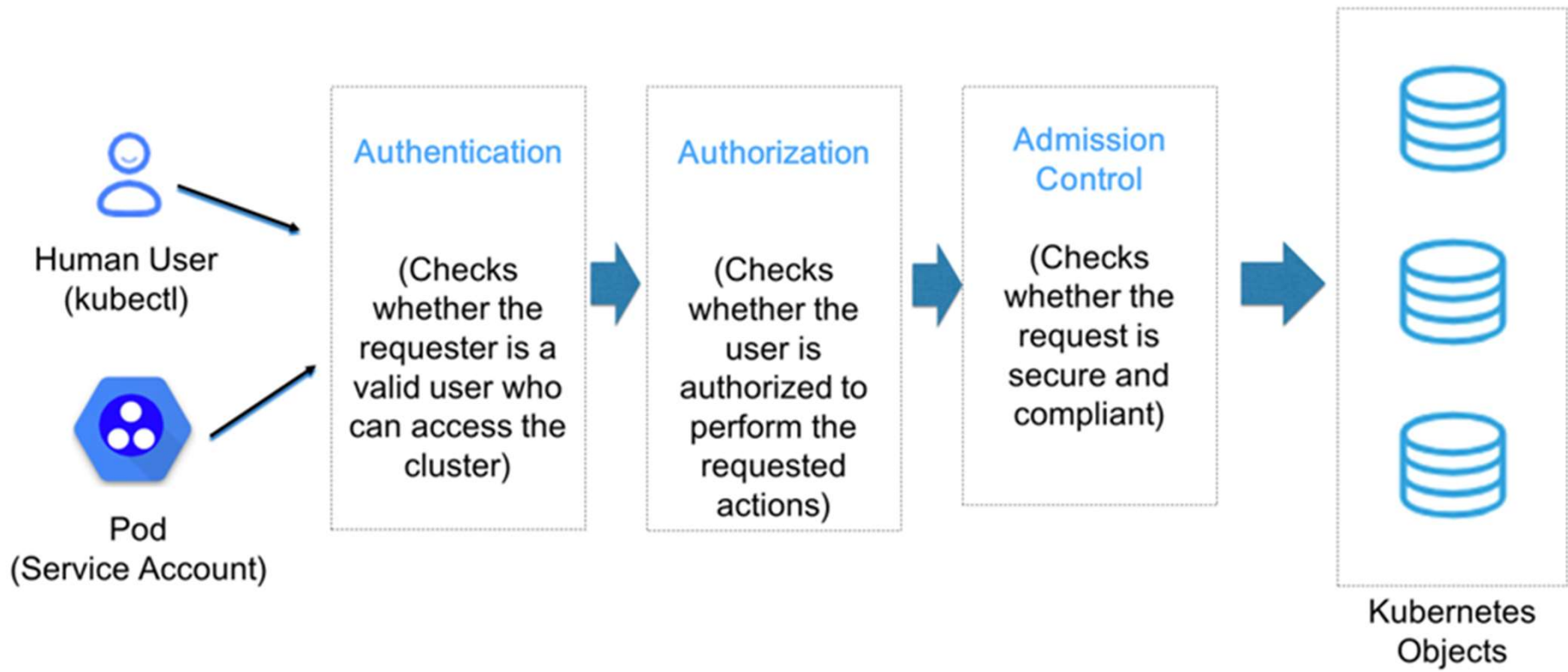
Each user or group of users have dedicated permissions in order to access the existing resources in the Kubernetes cluster, thus allowing us to have a complete control over the access mechanisms implemented inside the cluster.



K8s cluster



Kubernetes Authentication - Diagram





Beniefits of using RBAC

RBAC will :

- Have multiple users with different properties, establishing a proper authentication mechanism.
- Have full control over which operations each user or group of users can execute.
- Have full control over which operations each process inside a pod can execute.
- Limit the visibility of certain resources of namespaces.



Steps to implement RBAC

- Create a namespace
- Create users or groups
- Create a service account in case of applications.
- Define a role
- Define a Role Binding
- Test the access to cluster

Namespaces in Kubernetes



Let's first discuss, What are Namespaces ?

- Namespaces are Kubernetes objects which partition a single Kubernetes cluster into multiple virtual clusters.
- Namespaces impart a scope for object names within a cluster. While names must be unique within a namespace, the same name can be used in different namespaces.
- Command to create a namespace :

```
kubectl create namespace dev-env
```

Let's implement RBAC for a user



- All Kubernetes clusters have two categories of users:
 - service accounts managed by Kubernetes, and
 - normal users.
- Kubernetes does not have API Objects for User Accounts, so we will use X.509 client certificates with OpenSSL certificates to create a user.



Create User

The necessary steps are:

- Create a private key for your user

```
openssl genrsa -out user1.key 2048
```

- Create a certificate sign request using the private key

```
openssl req -new -key user1.key -out user1.csr -subj  
"/CN=user1/O=developer"
```

- Locate your Kubernetes cluster certificate authority (CA) which is responsible for approving the request and generating the necessary certificate to access the cluster API.

- Generate the final certificate by approving the certificate sign request

```
openssl x509 -req -in user1.csr -CA CA_LOCATION/ca.crt -CAkey  
CA_LOCATION/ca.key -CAcreateserial -out user1.crt -days 100
```

- CA_LOCATION is the location of your cluster CA. (default: /etc/kubernetes/pki/)
- It will be valid for 100 days.

Create User

... continued



- Add a new context with the new credentials for your Kubernetes cluster
- `kubect config set-credentials user1 --client-certificate=/home/ubuntu/.certs/user1.crt --client-key=/home/ubuntu/.certs/user1.key`
- `kubect config set-context user1 --context --cluster=minikube --namespace=dev-env --user=user1`

The user `user1` is created now in the developer group, and thier context is also set.



Defining a Role

- There are 2 types roles ,which represent a set of permissions (purely additive).
 - Role
 - ClusterRole
- A Role always sets permissions within a particular namespace.
- ClusterRole, by contrast, is a non-namespaced(cluster) resource.

```
role.yml
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: Role
3  metadata:
4    namespace: default
5    name: pod-reader
6  rules:
7  - apiGroups: [""] # "" indicates the core API group
8    resources: ["pods"]
9    verbs: ["get", "watch", "list"]
```




Defining a Role – api-groups

- You can specify other API groups. For example:
 - apps (for resources like deployments, statefulsets)
 - batch (for jobs, cronjobs)
 - rbac.authorization.k8s.io (for roles, rolebindings)

- **Example:**

apiGroups: ["apps"]

resources: ["deployments"]

Defining a Role – resources , verbs



- There are many resource types in Kubernetes.
Some common ones:
 - "pods", "services", "configmaps", "secrets", "deployments", "replicasets", "jobs", "namespaces", "nodes", etc.
- You can grant access to multiple resources in a single rule:

resources: ["pods", "services", "configmaps"]

Common Verbs:

- **get** – read a specific resource
- **list** – list resources
- **watch** – monitor changes
- **create** – create new resources
- **update** – update existing resources
- **patch** – apply partial modifications
- **delete** – delete resources
- **deletecollection** – delete a collection of resources
- **proxy** – access via the Kubernetes API proxy
- **impersonate** – act as another user/service account



Defining a Role – Grant ALL Example

```
- apiGroups: [""]  
  resources: ["pods"]  
  verbs: ["*"]
```



Defining a RoleBinding

- They allow us to bind the roles with users or Service Accounts.
- Based on the types of roles, we have different types of RoleBinding,
 - RoleBinding
 - ClusterRoleBinding

```
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: RoleBinding
3  metadata:
4    name: read-pods
5    namespace: default
6  subjects:
7  - kind: User
8    name: jane
9    apiGroup: rbac.authorization.k8s.io
10  roleRef:
11    kind: Role
12    name: pod-reader
13    apiGroup: rbac.authorization.k8s.io
```

For Applications

Your subtitle will be here



- Create a Service Account,

Pods that want to interact with the API server will authenticate with a particular **service account**.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tiller
  namespace: kube-system
```

- Create a Role,

A Role always sets permissions within a particular namespace.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: tiller-role
  namespace: kube-system
rules:
- apiGroups: ["", "extensions", "apps"]
  resources: ["*"]
  verbs: ["*"]
```

For an Application ... continued,

- Create a RoleBinding,

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: tiller-role-binding
  namespace: kube-system
subjects:
- kind: ServiceAccount
  name: tiller
  namespace: kube-system
roleRef:
  kind: Role
  name: tiller-role
  apiGroup: rbac.authorization.k8s.io
```

A role binding grants the permissions defined in a role to a user or set of users. It holds a list of *subjects* (users, groups, or service accounts), and a reference to the role being granted.



RBAC for a QA



Resources	Actions
pod	get
service	list
deployment	watch

Role : Tester

ROLE BINDING





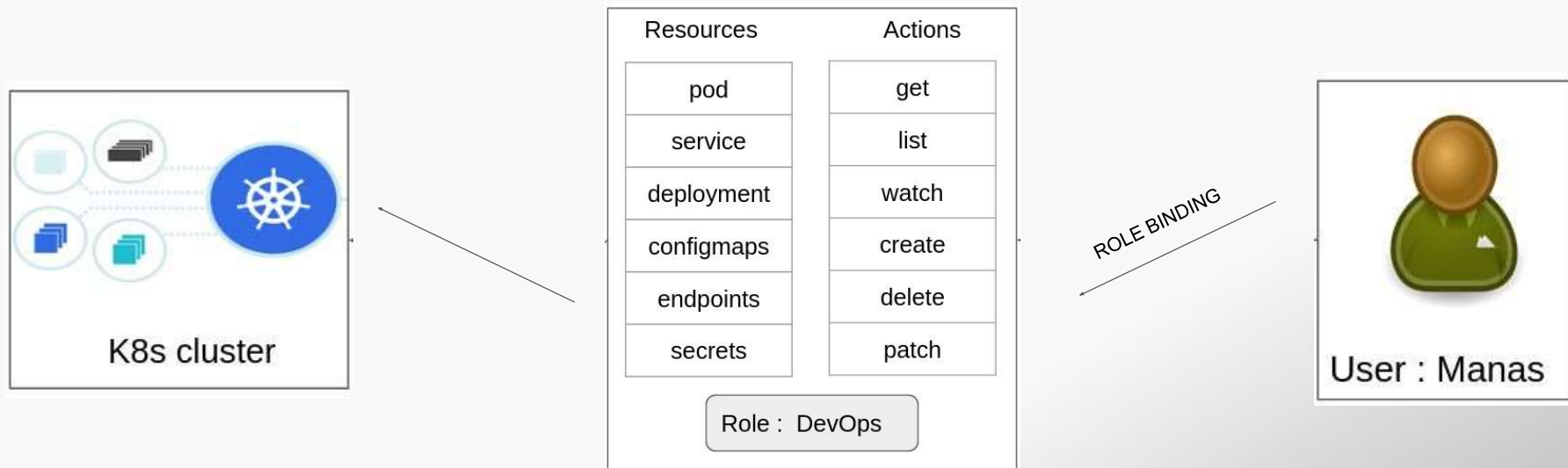
Let's test for access of QA

- kubectl provides the auth can-i subcommand for quickly querying the API authorization layer.
- Example:

```
kubectl auth can-i create pod --namespace prod --as sparsh  
No
```

```
kubectl auth can-i list pod --namespace prod --as sparsh  
Yes
```


RBAC for a DevOps Engineer





Let's test for the access of a DevOps Engineer

- We can use same kubectl with its subcommand auth can-i for querying the API authorization layer.
- Example:

```
kubectl auth can-i delete services --namespace prod --as manas  
Yes
```

```
kubectl auth can-i list secrets --namespace dev --as manas  
Yes
```



Agenda

- **Kubernetes Authentication**
- **Managing Users in Kubernetes**
- **Service Account**
- **Managing Roles and Role Binding**
- **Managing Cluster Role and Cluster Role Binding**
- **Basic Security Context**



Kubernetes Authentication

- **Purpose:** Determines *who* can access the Kubernetes API.
- **Mechanisms:**
 - **Client certificates** (most common for admins)
 - **Static token files** (for automation/service accounts)
 - **OpenID Connect (OIDC)** (for SSO/enterprise use)
 - **Webhook tokens**
- Kubernetes doesn't store user objects; relies on external identity.
- Authentication is the **first step**; *authorization* comes next.



Kubernetes Access Control

- Who
- Can perform what actions on what resources
- Under what conditions



Kubernetes resource model

ConfigMaps
Pod Service AutoScaler
Deployment Secrets
PV ReplicaSets Ingress
Namespace DaemonSet
CronJob Job PVC Nodes

